

Java Backend Interview Prep - Collections, Exception Handling, and String

This document answers the Collections, Exception Handling, and String topics listed in the source interview sheet.[file:1]

Collections

1) ArrayList internal working

ArrayList uses a resizable array. When capacity is exhausted, it creates a larger array and copies old elements into it.[file:1]

Key points:

- Fast random access: $O(1)$.
- Middle insertion/deletion is expensive due to shifting.
- Good default choice for read-heavy list usage.

2) HashMap internal working

HashMap stores entries in buckets based on hash. Index is derived from the key's hash and table size.[file:1]

Flow:

1. Compute `hashCode()`.
2. Spread/mix bits.
3. Find bucket index.
4. Compare keys using `equals()`.
5. Insert new node or replace existing value.

If the same key is put again, the old value is replaced.

Important interview note:

- Good `hashCode()` distributes keys.
- `equals()` decides logical equality.
- Heavy collisions first form linked structures; in modern JDKs, large buckets may become balanced trees.

3) ConcurrentHashMap vs Hashtable

Hashtable synchronizes almost every operation on the whole map, causing more contention. ConcurrentHashMap provides better concurrency using finer-grained synchronization/CAS mechanisms.[file:1]

Use ConcurrentHashMap when many threads read/write shared map data.

4) Vector vs ArrayList

- Vector is synchronized; ArrayList is not.
- ArrayList is usually faster in single-threaded or externally synchronized code.
- Vector is legacy; modern code usually prefers ArrayList or concurrent collections.

5) LinkedList vs ArrayList

LinkedList is useful when frequent insertions/deletions happen through iterators at known positions. ArrayList is better for indexing and cache-friendly access.

6) TreeMap vs HashMap

- HashMap: unordered, average $O(1)$ operations.
- TreeMap: sorted by natural order or comparator, usually $O(\log n)$.
- TreeMap is implemented using a Red-Black Tree.[file:1]

7) Immutable class

An immutable class cannot change state after construction. It is useful for thread safety, caching, predictable behavior, and safe map keys.[file:1]

Checklist:

- Make class final.
- Make fields private final.
- No setters.
- Defensive copies for mutable inputs/outputs.
- Validate in constructor.

```
public final class Employee {
    private final String name;
    private final List<String> skills;

    public Employee(String name, List<String> skills) {
        this.name = name;
        this.skills = List.copyOf(skills);
    }

    public String getName() { return name; }
```

```
    public List<String> getSkills() { return skills; }  
}
```

8) Shallow vs deep copy

- **Shallow copy** copies references.
- **Deep copy** copies the full object graph.

For immutable class design, deep defensive copying of mutable fields is often required.

9) Comparable vs Comparator

- **Comparable**: natural ordering, implemented inside the class.
- **Comparator**: external/custom ordering.

```
class Employee {  
    int id;  
    String name;  
    double salary;  
    int age;  
}  
  
Comparator<Employee> bySalaryDesc =  
    Comparator.comparingDouble((Employee e) -> e.salary).reversed();
```

10) CopyOnWriteArrayList

Best for read-heavy, write-light workloads. On each write, it creates a fresh internal copy.[file:1]

Pros:

- Safe iteration without `ConcurrentModificationException`.
- Excellent for subscriber lists or configuration snapshots.

Cons:

- Expensive writes.
- Higher memory overhead.

11) WeakHashMap, WeakReference, SoftReference

- **WeakReference**: collected eagerly when no strong refs remain.
- **WeakHashMap**: keys are weakly referenced.
- **SoftReference**: more GC-friendly cache style; collected under memory pressure.

12) ConcurrentModificationException

Usually happens when a collection is structurally modified during iteration by code that is not using the iterator's own `remove()`.

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().startsWith("x")) {
        it.remove();
    }
}
```

13) Why generic arrays are not allowed

Arrays are covariant and reified; generics are invariant and use type erasure. Mixing both would break type safety.

14) PriorityQueue

PriorityQueue is a heap-based queue where the head is the smallest element by default, or based on a custom comparator.

Exception handling

15) Base class for Error and Exception

Both Error and Exception extend Throwable.[file:1]

16) finally block and output example

finally executes whether exception occurs or not, mainly for cleanup. If finally contains a return, it overrides prior returns.[file:1]

Output:

```
In try block
In finally block
Result: 30
```

17) Checked vs unchecked exceptions

- **Checked:** verified at compile time, e.g. IOException, SQLException.
- **Unchecked:** subclasses of RuntimeException, e.g. NullPointerException, IllegalArgumentException.

Handling rule:

- Checked exceptions are handled or declared.

- Unchecked exceptions usually indicate programming or validation issues and are prevented or handled centrally.

18) finally without catch

Yes, try can be followed by finally without catch.

Cases where finally may not run:

- System.exit().
- JVM crash.
- Power failure / process kill.

19) Exception propagation

If a method does not handle an exception, it propagates to the caller.

```
class Demo {  
    static void c() {  
        throw new RuntimeException("boom");  
    }  
    static void b() { c(); }  
    static void a() { b(); }  
}
```

20) Overriding and exceptions

For checked exceptions, child methods cannot throw broader/new checked exceptions than the parent declaration. For unchecked exceptions, they can throw runtime exceptions.

21) Try-with-resources

Use it for AutoCloseable resources like streams, readers, JDBC connections, and statements. It ensures deterministic cleanup and suppresses close exceptions correctly.[file:1]

```
try (BufferedReader br = Files.newBufferedReader(path)) {  
    return br.readLine();  
}
```

String

22) Why String is immutable

String is immutable because its internal value cannot be changed after creation. This improves security, thread safety, string pooling, and hashcode caching.[file:1]

23) String literal vs new String

```
String s = new String("Shiv");  
String s1 = "Shiv";  
String s2 = "Shiv";
```

Typical interview answer:

- One object in string pool for "Shiv".
- One extra heap object for new String("Shiv").
- s1 and s2 refer to the same pooled object.

24) String interning

Interning means canonicalizing equal string values into the string pool.

```
String s1 = "Java";  
String s2 = "Java";  
String s3 = new String("Java");  
String s4 = s3.intern();  
  
System.out.println(s1 == s2); // true  
System.out.println(s1 == s3); // false  
System.out.println(s1 == s4); // true
```

25) concat output

```
String s = "Hello";  
s.concat("World");  
System.out.println(s); // Hello
```

Reason: String is immutable, so concat() returns a new object.

26) String vs StringBuffer vs StringBuilder

Order for mutation performance in single-threaded code:

- StringBuilder fastest
- StringBuffer slower because synchronized
- String slowest for repeated modifications because it creates new objects

27) toString() use

toString() provides a string representation of an object, useful for logging, debugging, and API responses.

